Федеральное агентство по рыболовству

Федеральное государственное бюджетное образовательное учреждение высшего образования

«Астраханский государственный технический университет»

Система менеджмента качества в области образования, воспитания, науки и инноваций

ООО «ДКС РУС» по международному стандарту ISO 9001:2015

ИНСТИТУТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И КОММУНИКАЦИЙ

КАФЕДРА АВТОМАТИЗИРОВАННЫХ СИСТЕМ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

ЗАЩИТА ИНФОРМАЦИИ

ОТЧЕТ

о выполнении лабораторной работы № 2

Алгоритм шифрования RSA

Выполнил:
студент гр. ЗИНРб-51

_____ Генералов В. В.

«_____» _____ 2026 г.

Преподаватель:

_____ Савоськин И. А.

«_____» _____ 2026 г.

Астрахань – 2026

ВВЕДЕНИЕ

Работа посвящена алгоритму RSA (по первым буквам фамилий авторов – Rivest, Shamir, Adleman) – асимметричной криптосистеме, в которой текст шифруется открытым ключом, а расшифровывается закрытым. Текст представляется кодами символов кодировки ASCII (American Standard Code for Information Interchange).

На практике RSA применяется в гибридных схемах для передачи сеансового ключа симметричного шифра и в электронной цифровой подписи. Стойкость алгоритма определяется трудностью разложения модуля на простые множители, поэтому применяются модули длиной от 2048 до 4096 бит. Отсюда актуальность изучения генерации ключей и проверки того, насколько быстро закрытый ключ восстанавливается по открытому при малых модулях.

Цель работы: «изучить и проанализировать работу криптографического алгоритма шифрования RSA. Написать программу, реализующую работу протокола».

Задание: «Написать на языке высокого уровня программу, которая бы реализовывала алгоритм шифрования/расшифрования RSA. Программа должна поддерживать как ручной ввод входных значений, так и случайный. Программа должна работать с любыми символами из кодировки ASCII».

В работе разработана программа на языке Python 3 с режимами генерации случайных ключей, ручного ввода ключей, шифрования и расшифрования текста ASCII, самопроверки и вычисления закрытого ключа по открытому. Работа программы подтверждена воспроизведением примера методических указаний (сообщение «CAB»), проверкой ручного ввода на восьми наборах значений, шифрованием и расшифрованием четырёх текстов при случайных ключах, прогоном всех кодов ASCII при разных модулях, а также измерением времени шифрования и вычисления закрытого ключа по открытому.

1 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Симметричные криптосистемы используют один секретный ключ, асимметричные – пару ключей: открытым ключом текст шифруется, закрытым расшифровывается. RSA относится к асимметричным криптосистемам.

Для генерации ключей выбираются большие различные простые числа p и q, вычисляются модуль $n = p \cdot q$ и значение функции Эйлера $\varphi = (p - 1)(q - 1)$. Закрытый показатель d выбирается случайным и взаимно простым с φ, то есть наибольший общий делитель (НОД) чисел d и φ равен единице: НОД(d, φ) = 1. Открытый показатель e находится из условия

$$e \cdot d \bmod \varphi = 1. \tag{1.1}$$

Открытый ключ – пара $\{e, n\}$, закрытый – пара $\{d, n\}$. При $p = q$ формула для φ неверна, поэтому простые числа должны различаться.

Текст разбивается на блоки, каждый блок – число $M(i)$ из ряда 0, 1, 2, …, $n - 1$. Шифрование выполняется открытым ключом, расшифрование – закрытым:

$$C(i) = M(i)^e \bmod n, \quad M(i) = C(i)^d \bmod n, \tag{1.2}$$

где

$C(i)$ – i-й блок шифртекста;

$M(i)$ – i-й блок открытого текста.

Корректность расшифрования следует из равенства $e \cdot d = 1 + k \cdot \varphi$, где k – целое число, и теоремы Эйлера $M^{\varphi} \equiv 1 \pmod{n}$ при НОД(M, n) = 1:

$$C^d \equiv M^{e \cdot d} = M \cdot (M^{\varphi})^k \equiv M \pmod{n}.$$

Для блоков M, кратных p или q, сравнение доказывается по модулям p и q и объединяется китайской теоремой об остатках. Эталоном для самопроверки программы служит пример методических указаний – сообщение «CAB». При A = 1, B = 2, C = 3 оно представляется блоками 3, 1, 2. Выбраны $p = 3$, $q = 11$ и $d = 3$, тогда $n = 33$, $(p - 1)(q - 1) = 20$ и $e = 7$, поскольку $7 \cdot 3 \bmod 20 = 1$. Открытым ключом {7, 33} блоки 3, 1, 2 шифруются в 9, 1, 29: $3^7 \bmod 33 = 9$, $1^7 \bmod 33 = 1$, $2^7 \bmod 33 = 29$. Закрытым ключом {3, 33} они расшифровываются обратно в 3, 1, 2: $9^3 \bmod 33 = 3$, $1^3 \bmod 33 = 1$, $29^3 \bmod 33 = 2$.

Зная p и q, противник вычисляет φ и $d = e^{-1} \bmod \varphi$, поэтому стойкость RSA определяется трудностью факторизации n. Программа опирается на вспомогательные алгоритмы: бинарное возведение в степень по модулю (***modpow***), расширенный алгоритм Евклида (***egcd***, ***modinv***) и пробное деление (***is_prime***, ***factor***).

2 ОПИСАНИЕ АЛГОРИТМА И ПРОГРАММЫ

2.1 Постановка задачи

Там, где методические указания не задают требований, приняты допущения:

1. Единица шифрования – один символ: блок M равен коду символа.
2. Порог $n \geq 256$ (***N_MIN***) взят с запасом: для принимаемых кодов 0–127 (***read_text***) хватило бы $n \geq 128$; минимальный допустимый $n = 7 \cdot 37 = 259$, в самопроверке $n = 265$.
3. «Большие простые» при генерации – простые из [100, 9999] (***P_MIN***, ***P_MAX***), при ручном вводе – не больше 10^{12} (***P_LIMIT***); простота проверяется пробным делением до $\sqrt{p}$ (***is_prime***) без вероятностных тестов.
4. Случайно генерируются только p, q, d; открытый текст всегда вводит пользователь.
5. Шифртекст – целые числа через пробел; расшифрование – по $\{d, n\}$ без p и q.
6. В учебном RSA блоки не дополняются: одинаковые символы дают одинаковые $C(i)$.
7. Текст вводится одной строкой, поэтому символ перевода строки (код 10) в него не входит; шифрование всех кодов 0–127 подтверждается самопроверкой (пункт «5»).

Из задания вытекают требования: ключи вычисляются по методическим указаниям (d выбирается случайно, e находится из условия (1.1)), воспроизводится пример $p = 3$, $q = 11$; текст шифруется открытым ключом $\{e, n\}$ и расшифровывается закрытым ключом $\{d, n\}$; ключи вводятся вручную с проверкой условий и сообщением об ошибке или генерируются случайно; принимаются любые символы ASCII, что проверяется прогоном всех кодов.

2.2 Описание алгоритма

Генерация ключей реализована функцией ***make_keys***: ***key_errors*** отвергает непростые p, q, случай $p = q$ и модуль $n = p \cdot q < 256$ (***N_MIN***), ***modinv*** находит $e = d^{-1} \bmod \varphi$ расширенным алгоритмом Евклида (***egcd***) и нормализует результат в диапазон $[1, \varphi - 1]$.

Шифрование выполняется посимвольно: ***encrypt_text*** берёт блок M, равный коду символа, ***encrypt_int*** проверяет условие $0 \leq M < n$ и вычисляет C по формуле (1.2). Функция ***decrypt_text*** восстанавливает M по той же формуле (***decrypt_int***) по ключу $\{d, n\}$ без p и q.

Возведение в степень выполняет функция ***modpow*** бинарным методом: показатель просматривается по битам, основание возводится в квадрат, при единичном бите результат умножается на основание, после каждого умножения берётся остаток по модулю. Вычислять M^e целиком нельзя: при четырёхзначных p, q это число из миллионов цифр.

Метод опирается на тождество, верное для любых целых a и b:

$$(a \cdot b) \bmod n = ((a \bmod n) \cdot (b \bmod n)) \bmod n.$$

Поэтому промежуточные значения не превышают n^2, а число умножений составляет около

$2 \cdot \log_2 e$. Восстановление закрытого ключа по открытому (функция ***recover_private_key***) повторяет генерацию: функция ***factor*** раскладывает $n = p \cdot q$ пробным делением до $\sqrt{n}$, затем вычисляются $\varphi = (p - 1)(q - 1)$ и $d = e^{-1} \bmod \varphi$ (функция ***modinv***).

Общая схема работы RSA (участники, ключи и передаваемые данные) приведена на рисунке 2.1.

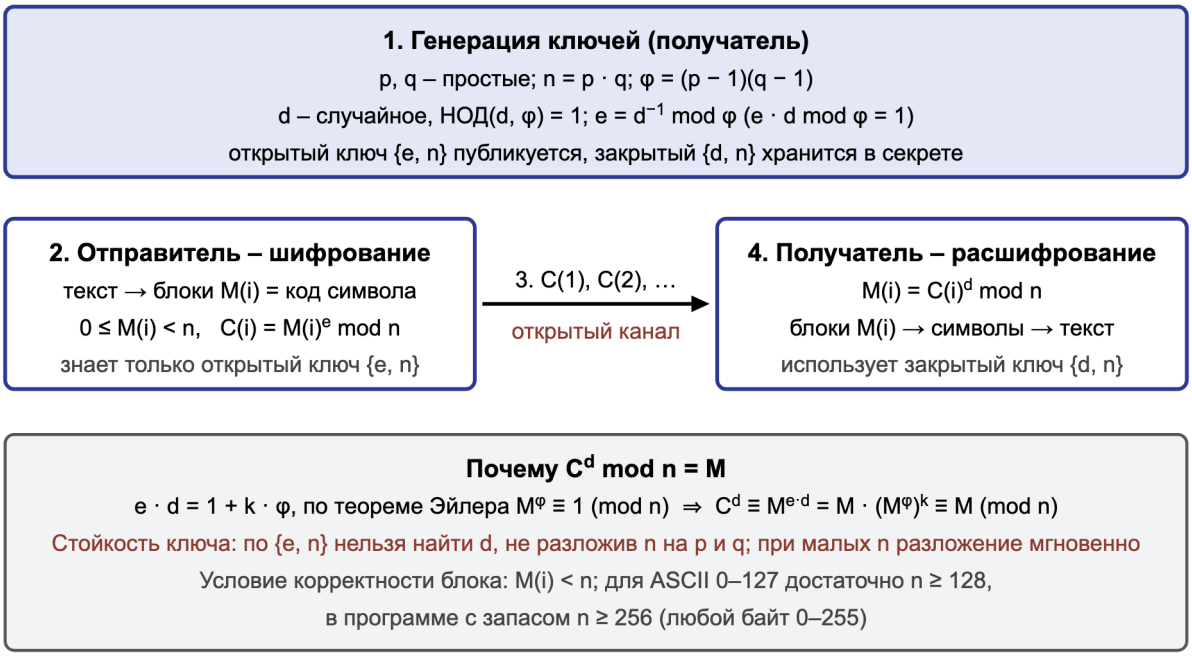


Рисунок 2.1 – Схема RSA: генерация ключей, шифрование открытым и расшифрование закрытым ключом

Получатель публикует $\{e, n\}$, отправитель шифрует символы этим ключом и передаёт числа $C(1)$, $C(2)$, …, получатель расшифровывает их ключом $\{d, n\}$.

2.3 Описание программы

Программа написана на Python 3 с использованием только стандартной библиотеки, состоит из одного модуля и запускается из консоли. Кроме описанных функций, в ней есть ***gen_prime***, ***gcd***, ***show_keys*** (вывод ключей), ***prime_error*** и ***small_n_error*** (сообщения проверки ручного ввода) и класс ***State*** с ключами и последним шифртекстом.

Меню программы (цикл ***main***) содержит пункты «1 — случайные ключи» (***random_keys***), «2 — ввод ключей вручную (p, q, d)» (***manual_keys***), «3 — зашифровать текст открытым ключом {e, n}» (***encrypt_mode***), «4 — расшифровать закрытым ключом {d, n}» (***decrypt_mode***), «5 — самопроверка (пример из методических указаний, все коды ASCII)» (***self_test***), «6 — вычислить закрытый ключ по открытому (факторизация n)» (***attack_mode***) и «0 — выход»; в пункте «4» клавиша «Enter» вместо ***d*** выбирает текущий закрытый ключ $\{d, n\}$ целиком, вместо шифртекста – последний шифртекст. Функция ***read_text*** отвергает символы с кодом больше 127 и повторяет запрос. При ручном вводе ключей функция ***read_int*** при ошибке выводит сообщение и повторяет запрос: «Нужно ввести целое число.», «p должно быть не больше 10^12: …», «p = 9

не простое», «q должно отличаться от p: …», «n = p·q = 33 < 256: …», «НОД(d, φ) = 4 ≠ 1: обратного e нет». Числа шифртекста, в том числе последнего, должны лежать в диапазоне [0, ***n*** − 1]. Если при неверном ключе или чужом шифртексте блок ***M*** не является кодом символа, функция ***decrypt_text*** возбуждает исключение ***ValueError***, и пункт «4» печатает «Не удалось расшифровать: …». Пункт «6» предупреждает, что разложение может занять много времени, и по «Ctrl+C» прерывает его с возвратом в меню; при вводе «Ctrl+D» и «Ctrl+C» завершают программу с сообщением «Выход.».

3 РЕЗУЛЬТАТЫ ТЕСТИРОВАНИЯ

3.1 Пример методических указаний

Пример «CAB» из методических указаний (***p*** = 3, ***q*** = 11, ***n*** = 33, ***d*** = 3, ***e*** = 7, «A» = 1, «B» = 2, «C» = 3) воспроизведён самопроверкой (пункт «5», функция ***self_test***). Кириллицы в ASCII нет, поэтому строки с кодами ASCII даны для латинских «C», «A», «B» и получены вызовом ***modpow***, так как ***encrypt_int*** отвергает $M \geq n$. Результаты приведены в таблице 3.1.

Таблица 3.1 – Пример «CAB» на числах и латинские буквы C, A, B на кодах ASCII

Блок ***M***	Ожидаемый ***C***	Полученный ***C***	***M'*** после расшифрования	Совпадение
«C» = 3	9	9	3	Да
«A» = 1	1	1	1	Да
«B» = 2	29	29	2	Да
«C» = 67 (ASCII)	–	1	1	Нет ($M \geq n$)
«A» = 65 (ASCII)	–	32	32	Нет ($M \geq n$)
«B» = 66 (ASCII)	–	0	0	Нет ($M \geq n$)

Блоки 3, 1, 2 дали $C = 9, 1, 29$ и вернулись без искажений, как в методических указаниях; коды 67, 65, 66 при $n = 33$ после расшифрования дали 1, 32, 0 = $M \bmod 33$, поскольку $M \geq n$.

3.2 Ручной ввод ключей

Ручной ввод (пункт «2», функция ***manual_keys***) проверен на восьми наборах значений; реакция программы приведена в таблице 3.2.

Таблица 3.2 – Ручной ввод ключей и реакция на некорректные значения

Ввод	Реакция программы
1. $p = 5$, $q = 53$, $d = 7$	Принято: $n = 265$, $e = 119$
2. $p = 61$, $q = 53$, $d = 17$	Принято: $n = 3233$, $e = 2753$
3. $p = 9$ (составное)	Сообщение «p = 9 не простое…»
4. $q = p = 11$	Сообщение «q должно отличаться от p…»
5. $p = 3$, $q = 11$ ($n = 33 < 256$)	Сообщение «n = p·q = 33 < 256…»
6. $p = 3$, $q = 43$ ($n = 129 < 256$)	Сообщение «n = p·q = 129 < 256…»
7. $p = 61$, $q = 53$, $d = 4$ (НОД(4, φ) ≠ 1)	Сообщение «НОД(d, φ) = 4 ≠ 1…»
8. d = «abc»	Сообщение «Нужно ввести целое число…»

Два корректных набора приняты и прошли цикл шифрования и расшифрования; составное

$p = 9$, $q = p$, $n = 33$ и $n = 129$, НОД$(d, \varphi) = 4$ и нечисловое значение «abc» отвергнуты с сообщением и повторным запросом.

3.3 Случайные ключи

Случайные ключи (пункт «1», функция ***random_keys***) проверены на четырёх текстах, включая все печатные символы с кодами 32–126 и пустую строку; результаты приведены в таблице 3.3.

Таблица 3.3 – Шифрование и расшифрование при случайных ключах

Текст	p, q	d, e	Начало шифртекста	Верно
1. «Hello, RSA 2026!» – буквы, цифры, знаки, пробел	6991 6143	39306859 25124419	10947667 10311295 1210064 1210064 …	Да
2. «aaa bbb» – повторы символов	4337 7951	31501307 2504243	6333550 6333550 6333550 24314473 …	Да
3. Все печатные символы 32–126 (95 символов)	4507 251	69437 763373	593251 109247 754828 849225 …	Да
4. «» – пустая строка	7607 9239	4512395 1685571	–	Да

Все тексты расшифрованы верно; одинаковые символы дают одинаковые $C(i)$ (6333550 для «a»). Полный сеанс работы (пункты «1», «3», «4», «6») показан на рисунке 3.1.

```
Выбор: 1
Случайные ключи: p, q — простые из [100, 9999], d — случайное, взаимно простое с φ.
  p = 4651, q = 9311, n = p·q = 43305461, φ = (p−1)(q−1) = 43291500
  d = 206879 (НОД(d, φ) = 1), e = d⁻¹ mod φ = 13175219, проверка: e·d mod φ = 1
  Открытый ключ {e, n} = {13175219, 43305461}, закрытый ключ {d, n} = {206879,
      43305461}
Выбор: 3
  Текст (ASCII): Hello, RSA 2026!
  Коды символов M: [72, 101, 108, 108, 111, 44, 32, 82, 83, 65, 32, 50, 48, 50, 54, 33]
  Шифртекст C = M^e mod n ({e, n} = {13175219, 43305461}):
  30447288 5484913 3784637 3784637 18324008 7274101 29857261 27537519 21733373
      29127549 29857261 39050216 16534966 39050216 3532151 13865455
Выбор: 4
  d (Enter — текущий закрытый ключ):
  Шифртекст, числа через пробел (Enter — последний результат):
  M = C^d mod n ({d, n} = {206879, 43305461}): [72, 101, 108, 108, 111, 44,
      32, 82, 83, 65, 32, 50, 48, 50, 54, 33]
  Расшифрованный текст: 'Hello, RSA 2026!'
Выбор: 6
  Противник знает только открытый ключ {e, n} = {13175219, 43305461}.
  Разложение n пробным делением может занять много времени (Ctrl+C — прервать).
  Факторизация: n = 4651 · 9311, φ = 43291500, d = e⁻¹ mod φ = 206879  (0.107 мс)
  Совпадает с настоящим закрытым ключом: да
```

Рисунок 3.1 – Сеанс: ключи, шифрование, расшифрование, вычисление закрытого ключа

В сеансе расшифрование выполнено по ключу $\{d, n\}$ без p и q, а закрытый ключ, восстановленный по открытому за 0,107 мс, совпал с настоящим.

3.4 Охват кодов символов

Доля кодов 0–127 и 0–255, верно проходящих цикл шифрования и расшифрования через ***modpow*** при n = 33, 129, 265, 10403, 1005973, приведена в таблице 3.4.

Таблица 3.4 – Доля кодов 0–127 (ASCII) и 0–255, верно проходящих цикл, при разных n

Разложение n	n	Верно из кодов 0–127	Верно из кодов 0–255	Условие $n \geq 256$
3·11	33	33/128	33/256	Нет
3·43	129	128/128	129/256	Нет
5·53	265	128/128	256/256	Да
101·103	10403	128/128	256/256	Да
997·1009	1005973	128/128	256/256	Да

Экран самопроверки (пункт «5») с итогом «OK» показан на рисунке 3.2.

```
Лабораторная работа №2. Алгоритм шифрования RSA
  1 — случайные ключи
  2 — ввод ключей вручную (p, q, d)
  3 — зашифровать текст открытым ключом {e, n}
  4 — расшифровать закрытым ключом {d, n}
  5 — самопроверка (пример из методических указаний, все коды ASCII)
  6 — вычислить закрытый ключ по открытому (факторизация n)
  0 — выход
Выбор: 5
  Пример методических указаний: p=3, q=11, n=33, φ=20, d=3 → e=7 (ожидалось 7)
    M = [3, 1, 2] → C = [9, 1, 29] (ожидалось [9, 1, 29]) → M' = [3, 1, 2]: PASS
  Все коды 0..255 через ключ n=265 (p=5, q=53, d=7, e=119): совпало 256/256: PASS
  Итог самопроверки: OK
```

Рисунок 3.2 – Самопроверка: пример методических указаний и все коды ASCII

Пример методических указаний пройден, через ключ $n = 5 \cdot 53 = 265$ цикл проходят все 256 кодов из 256. Таким образом, при n = 33 верны 33/128 кодов 0–127, при n = 129 – все 128/128, но из кодов 0–255 только 129/256, при $n \geq 265$ – все 256/256.

3.5 Быстродействие и стойкость

Время шифрования 100 символов при наивном вычислении $(M^e) \bmod n$ целиком и при бинарном методе ***modpow*** приведено в таблице 3.5: наивное возведение заняло 1,24 и 32 100 мс и было прервано при четырёхзначных p, q; ***modpow*** – от 0,09 до 0,23 мс.

Таблица 3.5 – Время шифрования 100 символов

Разрядность p, q	n	e	Возведение целиком, мс	***modpow***, мс
Двузначные	3233	899	1,24	0,09
Трёхзначные	988027	603263	32 100	0,22
Четырёхзначные	99400891	18877835	Более 10 000 на символ (прервано)	0,23

Закрытый ключ по открытому (пункт «6», функция ***recover_private_key***) вычислен факторизацией ***n*** для ***n*** ≈ 10^4, 10^6, 10^8; результаты приведены в таблице 3.6.

Таблица 3.6 – Вычисление закрытого ключа по открытому ключу {*e*, *n*}

n	***e***	Разложение ***p*·*q***	Время, мс	Восстановленный ***d***	Совпадение ***d***
10403	8743	101·103	0,004	7	Да
1005973	547619	997·1009	0,023	11	Да
99400891	14197279	9967·9973	0,213	7	Да

Факторизация заняла от 0,004 до 0,213 мс, ***d*** восстановлен во всех случаях.

3.6 Анализ результатов

Требования задания подтверждены: реализация шифрования и расшифрования RSA – таблицами 3.1 и 3.3 и рисунками 3.1 и 3.2, поддержка ручного и случайного ввода входных значений – таблицами 3.2 и 3.3, работа с любыми символами кодировки ASCII – таблицей 3.4.

Пример методических указаний воспроизводится только в числовом режиме: при кодах ASCII и ***n*** = 33 расшифрование даёт ***M*** mod ***n*** (таблица 3.1). Порог ***N_MIN*** = 256 взят с запасом: ***n*** = 129 хватает для кодов 0–127, но не для 0–255 (таблица 3.4); минимальный допустимый ***n*** = 259.

Проверка ключей обязательна: при составном ***p*** или ***p*** = ***q*** величина (***p*** − 1)(***q*** − 1) не равна ***φ***(***n***), и равенство $C^d \bmod n = M$ не гарантировано, поэтому ***manual_keys*** отвергает такие ***p***, ***q*** (таблица 3.2). Условие НОД(***d***, ***φ***) = 1 обеспечивает существование ***e***: ***make_keys*** подбирает случайное ***d*** в цикле, неподходящее введённое ***d*** отвергается.

Из таблицы 3.5 следует, что вычислять M^e целиком нельзя: при трёхзначных ***p***, ***q*** – 32 100 мс на 100 символов, при четырёхзначных – более 10 000 мс на символ; ***modpow*** укладывается в 0,23 мс. Стойкость на учебных размерах отсутствует: ***n*** ≈ 10^8 факторизуется пробным делением за 0,213 мс, и ***d*** восстанавливается из открытого ключа (таблица 3.6); реальные модули имеют длину от 2048 до 4096 бит.

Ограничение реализации – посимвольный детерминированный шифр без дополнения блоков: одинаковые символы дают одинаковые ***C*** (таблица 3.3, текст «aaa bbb»), что допускает вскрытие по словарю из 128 зашифрованных кодов.

4 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

Вопрос 1. Чем отличаются симметричные и асимметричные криптосистемы? К какой относится алгоритм RSA?

В симметричной системе обе стороны используют один секретный ключ, который приходится передавать по защищённому каналу. В асимметричной системе ключей два: открытый $\{e, n\}$ публикуется, закрытый $\{d, n\}$ хранится у владельца; стойкость опирается на трудную математическую задачу. RSA – асимметричная криптосистема, основанная на трудности факторизации n. На практике применяют гибрид: сеансовый ключ передаётся зашифрованным по RSA или вырабатывается сторонами по протоколу Диффи–Хеллмана (лабораторная работа № 1), а данные шифрует симметричный шифр.

Вопрос 2. Можно ли вычислить закрытый ключ, зная открытый?

Формально да: разложив n на множители p и q, противник вычисляет $\varphi = (p - 1)(q - 1)$ и $d = e^{-1} \bmod \varphi$. Так действует пункт меню «6» (функция ***recover_private_key***): учебные n до 10^8 факторизуются за доли миллисекунды, d восстанавливается точно (таблица 3.6, см. рис. 3.1). Однако эффективного алгоритма факторизации не известно, и для применяемых на практике n длиной 2048 бит такое вычисление невыполнимо. При правильно выбранных p и q стойкость к вычислению закрытого ключа определяется прежде всего размером n; посимвольное шифрование без дополнения, как в данной программе, вскрывается и без факторизации – сопоставлением с 128 блоками, зашифрованными открытым ключом.

Вопрос 3. Может ли алгоритм RSA использоваться в электронной цифровой подписи?

Да. Как отмечено в методических указаниях, RSA стал первым алгоритмом с открытым ключом, пригодным и для шифрования, и для цифровой подписи, и остаётся одним из стандартных алгоритмов подписи. При подписи роли ключей меняются местами. Отправитель вычисляет хэш-значение сообщения h и подписывает хэш-значение закрытым ключом: $S = h^d \bmod n$; получатель открытым ключом проверяет равенство $S^e \bmod n = h$. Подписывается хэш-значение, а не само сообщение: объём вычислений не зависит от длины сообщения. Хэш-значение должно быть меньше n – то же правило $M < n$, что и при шифровании. Подпись обеспечивает целостность, аутентичность и неотказуемость (теоретические сведения к лабораторной работе № 4).

ЗАКЛЮЧЕНИЕ

В работе разработана консольная программа на языке Python 3, реализующая алгоритм RSA: случайная генерация ключей, ручной ввод p, q, d, шифрование, расшифрование по ключу $\{d, n\}$, самопроверка и вычисление закрытого ключа по открытому.

Все требования задания выполнены. Шифрование и расшифрование воспроизводят пример методических указаний с сообщением «CAB» (блоки 3, 1, 2 → 9, 1, 29 → 3, 1, 2 при ключах $\{7, 33\}$ и $\{3, 33\}$); из восьми наборов ручного ввода два корректных приняты, шесть некорректных отвергнуты с сообщением; четыре текста при случайных ключах восстановлены верно; при $n \geq 256$ цикл шифрования и расшифрования проходят все 256 кодов 0–255, в том числе все коды ASCII 0–127, то есть программа работает с любыми символами ASCII.

Корректность алгоритма требует простых $p \neq q$, условий $n > \max M$ и НОД$(d, \varphi) = 1$, а практическая выполнимость – быстрого возведения в степень по модулю (***modpow***). Учебные ключи с четырёхзначными p, q стойкости не имеют: ***recover_private_key*** факторизует n и находит d за доли миллисекунды.

Ограничения реализации – посимвольный детерминированный шифр без дополнения блоков (одинаковые символы дают одинаковые блоки шифртекста, и текст вскрывается по словарю из 128 зашифрованных кодов) и учебный размер модуля, тогда как на практике применяются модули длиной от 2048 до 4096 бит.